

Gerenciamento alunos Desenvolvimento Java

José Nilson Oliveira Arcanjo
Larissa Bonfim de Jesus

Salvador-BA
2025

Sumário

1. Introdução
2. Camada Model
3. Camada Service
4. Camada Controller
5. Camada Repository
- 6.HTML
- 7.Site

1. Introdução

Nesse Projeto decidimos criar um gerenciamento de alunos para uma escola, facilitando as mesmas a ter um melhor controle da quantidade de alunos por turma e realizar seus cadastros, nessa API existem funções como cadastrar, atualizar, deletar e buscar por turma.

2. Camada Model

```
@Entity
@Table(name = "tb_user")
public class Aluno {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(columnDefinition = "TEXT")
    private String nome;
    private Integer turma;

    public Aluno() {
    }

    public Aluno(Aluno aluno) {
        this.id = aluno.getId();
        this.nome = aluno.getNome();
        this.turma = aluno.getTurma();
    }

    public Aluno(Long id, String nome, Integer turma) {
        this.id = id;
        this.nome = nome;
        this.turma = turma;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    public Integer getTurma() {
        return turma;
    }

    public void setTurma(Integer turma) {
        this.turma = turma;
    }
}
```

Na classe aluno é possível ver o banco de dados sendo criado através de anotações como @Table, além da definição de um id, por meio de @Id.

Nessa classe é definida os elementos do cadastro como nome e turma. Com eles definidos é feito os gets e sets.

3. Camada Service

```
@Service
public class AlunoService {

    @Autowired
    AlunoRepository alunoRepository;

    @Transactional(readOnly = true)
    public ResponseEntity<> findAllTurma(int turma){
        List<AlunoProjection> AlunoProjection = alunoRepository.searchAllAlunosTurma(turma);

        return ResponseEntity.status(HttpStatus.OK).body(AlunoProjection);
    }

    @Transactional(readOnly = true)
    public ResponseEntity<> findAllAlunos(){
        List<Aluno> aluno = alunoRepository.findAll();

        return ResponseEntity.status(HttpStatus.OK).body(aluno);
    }

    @Transactional
    public ResponseEntity<> saveAluno(Aluno aluno){
        alunoRepository.save(new Aluno(aluno));
        return ResponseEntity.status(HttpStatus.CREATED).body(new ResponseMessage("Aluno criado com sucesso."));
    }

    @Transactional
    public ResponseEntity<> updateAluno(Long id, Aluno aluno){
        Optional<Aluno> alunoOptional = alunoRepository.findById(id);

        if (!alunoOptional.isPresent()) {
            return ResponseEntity.status(HttpStatus.NOT_FOUND).body(new ResponseMessage("Aluno não encontrado."));
        }

        Aluno result = alunoOptional.get();

        result.setNome(aluno.getNome());
        result.setTurma(aluno.getTurma());

        alunoRepository.save(result);
        return ResponseEntity.status(HttpStatus.ACCEPTED).body(new ResponseMessage("Aluno alterado com sucesso."));
    }

    @Transactional
    public ResponseEntity<> deleteAluno(Long id){
        Optional<Aluno> alunoOptional = alunoRepository.findById(id);

        if (!alunoOptional.isPresent()) {
            return ResponseEntity.status(HttpStatus.NOT_FOUND).body(new ResponseMessage("Aluno não encontrado."));
        }

        alunoRepository.deleteById(id);

        return ResponseEntity.status(HttpStatus.OK).body(new ResponseMessage("Aluno excluído com sucesso."));
    }
}
```

A classe AlunoService contém toda a lógica de negócio. Ela usa @Transactional para garantir consistência nas operações e acessa o banco através do repository. Nessa camada foram criadas algumas funcionalidades como:

- findAllTurma: retorna lista de alunos filtrados pela turma.
- findAllAlunos: retorna todos os alunos.
- saveAluno: cria um novo aluno.
- updateAluno: atualiza informações de um aluno existente.
- deleteAluno: exclui aluno.

4. Camada Controller

```
@RestController
@RequestMapping(value = "/users")
public class AlunoController {

    @Autowired
    AlunoService alunoService;

    @GetMapping
    public ResponseEntity<> findAllAlunos(){
        return alunoService.findAllAlunos();
    }

    @GetMapping(value = "/turma/{turma}")
    public ResponseEntity<> findAllTurma(@PathVariable int turma){
        return alunoService.findAllTurma(turma);
    }

    @PostMapping
    public ResponseEntity<> saveAluno(@RequestBody Aluno aluno){
        return alunoService.saveAluno(aluno);
    }

    @PutMapping(value =("/{id}")
    public ResponseEntity<> updateAluno(@PathVariable Long id, @RequestBody Aluno aluno){
        return alunoService.updateAluno(id, aluno);
    }

    @DeleteMapping(value =("/{id}")
    public ResponseEntity<> deleteAluno(@PathVariable Long id){
        return alunoService.deleteAluno(id);
    }
}
```

A camada Controller é responsável por receber as requisições do cliente, funcionando como porta de entrada e direcionando as requisições a camada service.

Na camada controller podemos encontrar anotações ligadas ao banco de dados

- RequestMapping/users: endpoint base do controller.
- GET /turma/{turma}: lista alunos filtrados pela turma.
- PostMapping: endpoint para salvar novo aluno.
- PutMapping /{id}: endpoint para atualiza aluno.
- DeleteMapping/{id}: endpoint para exclui aluno.

5. Camada Repository

```
1 package com.project.projectjavaJL.repositories;
2
3 import java.util.List;
4
5 import org.springframework.data.jpa.repository.JpaRepository;
6 import org.springframework.data.jpa.repository.Query;
7 import org.springframework.stereotype.Repository;
8
9 import com.project.projectjavaJL.model.Aluno;
10 import com.project.projectjavaJL.projections.AlunoProjection;
11
12 @Repository
13 public interface AlunoRepository extends JpaRepository<Aluno, Long> {
14     @Query(nativeQuery = true, value = """
15         SELECT tb_user.id, tb_user.nome, tb_user.turma
16         FROM tb_user
17         WHERE tb_user.turma = :turma
18         ORDER BY tb_user.turma
19         """)
20     List<AlunoProjection> searchAllAlunosTurma(int turma);
21 }
```

A camada Repository é responsável por fazer a comunicação com o banco de dados. Ela é quem realmente faz: consultas (SELECT), inserções (INSERT), atualizações (UPDATE), exclusões (DELETE).

6. HTML

```
<div class="container">
  <h1>Sistema de Gerenciamento de Alunos</h1>

  <div id="message-container" class="message-box" style="display: none;"></div>

  <h2>🔍 Pesquisar Alunos por Turma</h2>
  <div class="form-group" style="display: flex; gap: 10px;">
    <input type="number" id="searchTurma" placeholder="Digite o número da Turma" style="flex-grow: 1;">
    <button class="btn-info" onclick="searchAlunosByTurma()">Buscar Turma</button>
  </div>
  <div id="searchResult" style="margin-top: 10px;"></div>
</div>

<hr>

<h2 id="form-title">+ Cadastrar Novo Aluno</h2>
<form id="alunoForm">
  <input type="hidden" id="alunoId">
  <div class="form-group">
    <label for="nome">Nome:</label>
    <input type="text" id="nome" required>
  </div>
  <div class="form-group">
    <label for="turma">Turma:</label>
    <input type="number" id="turma" required>
  </div>
  <button type="submit" id="submitBtn" class="btn-primary">Cadastrar</button>
  <button type="button" id="cancelEditBtn" class="btn-warning" style="display: none;" onclick="resetForm()">Cancelar Edição</button>
</form>

<hr>

<h2>📋 Alunos Existentes</h2>
<table>
  <thead>
    <tr>
      <th>ID</th>
      <th>Nome</th>
      <th>Turma</th>
      <th>Ações</th>
    </tr>
  </thead>
  <tbody id="alunosTableBody">
  </tbody>
</table>
</div>
```

Funções para pesquisar, cadastrar e visualizar alunos existentes.

```
/**
 * Exibe uma mensagem de feedback (sucesso ou erro) para o usuário.
 */
function showMessage(message, isError = false) {
  messageContainer.style.display = 'block';
  messageContainer.textContent = message;
  messageContainer.className = 'message-box ' + (isError ? 'error' : 'success');
  setTimeout(() => {
    messageContainer.style.display = 'none';
  }, 5000);
}

/**
 * Limpa e reseta o formulário para o modo de Cadastro.
 */
function resetForm() {
  form.reset();
  alunoIdInput.value = '';
  formTitle.textContent = '+ Cadastrar Novo Aluno';
  submitBtn.textContent = 'Cadastrar';
  submitBtn.className = 'btn-primary';
  cancelEditBtn.style.display = 'none';
}
```

Avisa ao usuário que o cadastro foi concluído ou não e limpa o campo para um novo cadastro.

```

/**
 *Listar Todos os Alunos (GET /users)
 */
async function fetchAlunos() {
  try {
    const response = await fetch(BASE_URL);
    if (!response.ok) {
      throw new Error('Falha ao buscar alunos: ' + response.status);
    }
    const alunos = await response.json();
    renderTable(alunos);
  } catch (error) {
    console.error('Erro ao buscar alunos:', error);
    showMessage('Não foi possível carregar a lista de alunos.', true);
  }
}

/**
 *Cadastrar Novo Aluno (POST /users)
 */
async function createAluno(alunoData) {
  try {
    const response = await fetch(BASE_URL, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(alunoData)
    });

    const data = await response.json();
    if (!response.ok) {
      throw new Error(data.message || 'Erro ao cadastrar.');
    }

    showMessage(data.message || 'Aluno criado com sucesso!', false);
    resetForm();
    fetchAlunos(); // Atualiza a tabela
  } catch (error) {
    console.error('Erro na criação:', error);
    showMessage(error.message, true);
  }
}

```

Funções para criar e atualizar alunos.


```

/**
 * Excluir Aluno (DELETE /users/{id})
 */
async function deleteAluno(id) {
  if (!confirm('Tem certeza que deseja excluir o aluno com ID ${id}?')) {
    return;
  }

  try {
    const response = await fetch(`${BASE_URL}/${id}`, {
      method: 'DELETE'
    });

    const data = await response.json();
    if (!response.ok) {
      throw new Error(data.message || 'Erro ao excluir.');
```

Funções para deletar e listar os alunos pela sua turma.

7. Site

Acessado por: http://localhost:8088/aluno.html

 **Pesquisar Alunos por Turma**

Buscar Turma

 **Alunos encontrados na Turma 6ª:**

- ID: 1 | Nome: Juju Cardoso
- ID: 2 | Nome: Julia Cardoso
- ID: 3 | Nome: Maria Santos
- ID: 6 | Nome: Larissa Bomfim

 **Cadastrar Novo Aluno**

Nome:

Turma:

Cadastrar

 **Alunos Existentes**

ID	Nome	Turma	Ações
1	Juju Cardoso	6	EditarExcluir
2	Julia Cardoso	6	EditarExcluir
3	Maria Santos	6	EditarExcluir
4	Camila Vieira	9	EditarExcluir
5	José Nilson	7	EditarExcluir
6	Larissa Bomfim	6	EditarExcluir
7	Afonso Pereira	9	EditarExcluir
8	Maria Silva	7	EditarExcluir
9	José Santos	7	EditarExcluir